INTERNET OF THINGS

DISUSUN OLEH:

Nama : Salwa Fazirah
Kelas : 3B
NPM : 25782054

Dosen:

Agus Ambarwari, S.Pd., M.Kom.

Dr. Ir. Septafiansyah Dwi Putra,

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2025/2026

BAB I:
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi *Internet of Things* (IoT) memungkinkan berbagai perangkat elektronik untuk saling berkomunikasi dan bertukar data melalui jaringan internet. Cip NodeMCU ESP8266 hadir sebagai solusi mikrokontroler yang sudah dilengkapi dengan modul Wi-Fi internal, sehingga sangat fleksibel untuk diterapkan dalam arsitektur IoT. Untuk membangun komunikasi data antar perangkat, protokol HTTP (*HyperText Transfer Protocol*) menjadi salah satu fondasi utama. Komunikasi ini berjalan melalui skema *Client-Server* dengan metode seperti HTTP GET. Pemahaman mengenai cara mengkonfigurasi NodeMCU sebagai *Web Server* maupun *HTTP Client* sangat penting untuk merancang sistem pemantauan dan pengendalian jarak jauh secara terintegrasi

1.2 TUJUAN PRAKTIKUM

- Memahami perbedaan mode jaringan pada ESP8266 (*Station*, *Access Point*, dan *Dual Mode*).
- Mampu mengkonfigurasi NodeMCU ESP8266 sebagai **Web Server** untuk menyajikan antarmuka HTML tertanam (*embedded HTML*).
- Mampu mengkonfigurasi NodeMCU ESP8266 sebagai **HTTP Client** untuk mengirimkan instruksi HTTP GET ke *server* lain.
- Memahami implementasi arsitektur *Client-Server* menggunakan metode interaksi HTTP GET.

1.3Alat Dan Bahan

- Board NodeMCU ESP8266 (2 buah: 1 sebagai Server, 1 sebagai Client)
- Cable Micro USB (2 buah)
- Breadboard (2 buah)
- Sensor Suhu & Kelembapan DHT11 / DHT22 (1 buah)
- Sensor LDR + Resistor 10k Ohm (1 buah)
- Modul Relay 1-Channel 5V (1 buah)
- Kabel *Jumper Male-to-Male* & *Male-to-Female* (Secukupnya)
- Smartphone (Sebagai Wi-Fi Hotspot/Tethering) & Laptop/Browser

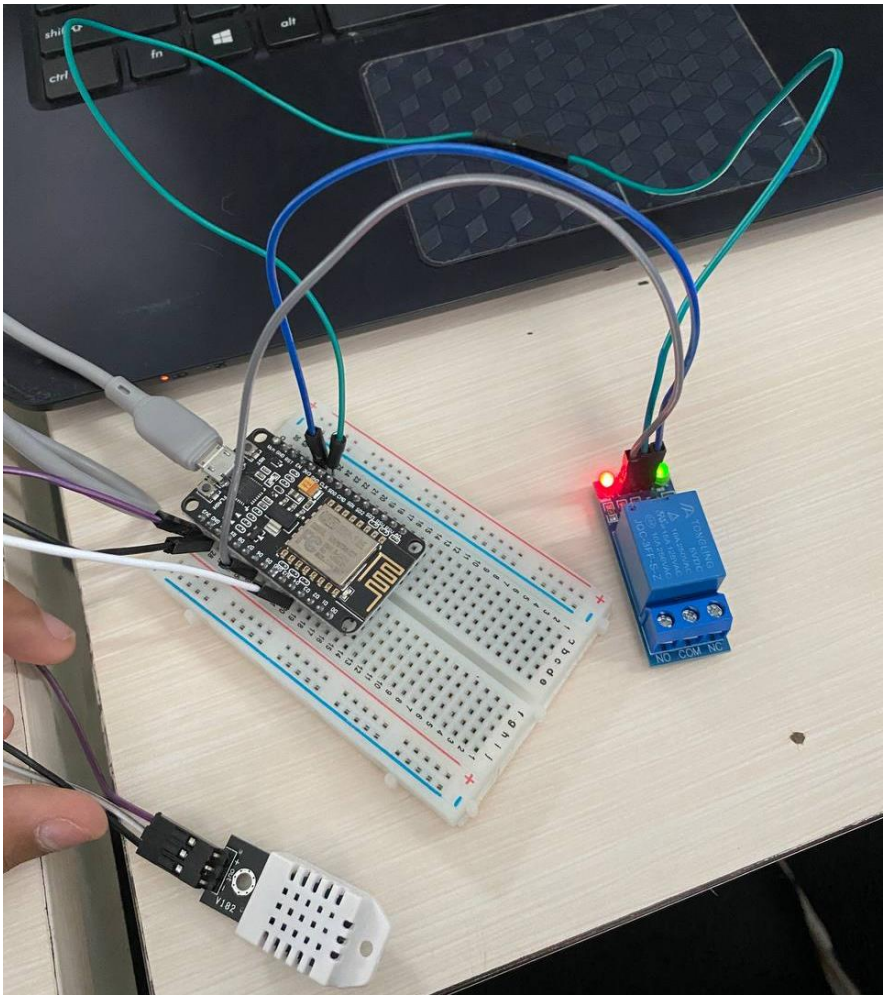
BAB II

Hasil Dan Pembahasan Praktikum

2.1 Praktikum 1: NodeMCU sebagai Web Server (Menyajikan HTML)

Rangkaian & Skematik

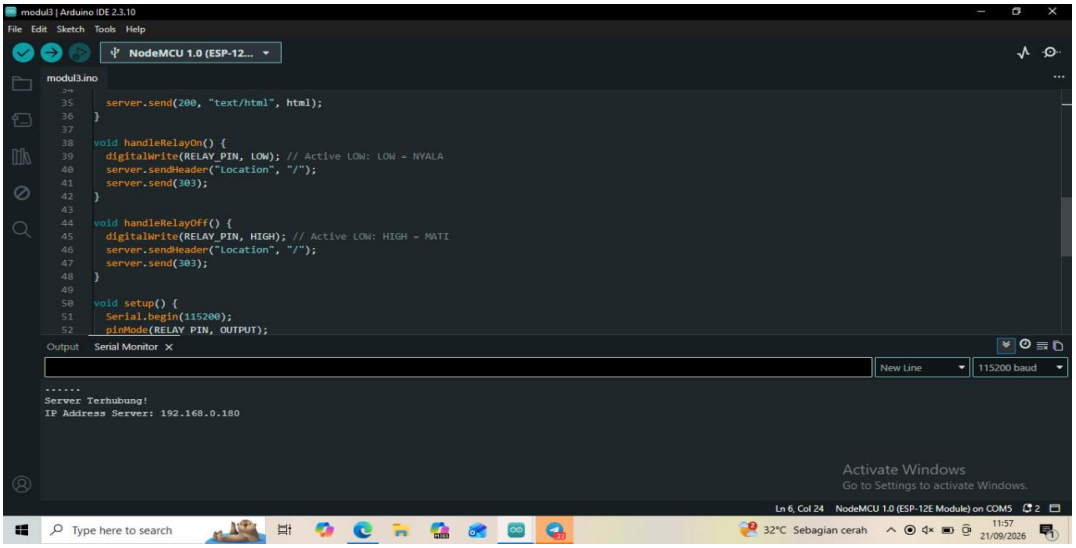
- **DHT11 Data Pin:** Terhubung ke Pin **D4 (GPIO 2)** NodeMCU. VCC ke 3V3, GND ke GND.
- **Relay Signal Pin:** Terhubung ke Pin **D6 (GPIO 12)** NodeMCU. VCC ke VIN/VU, GND ke GND.



(Foto skematik rangkaian NodeMCU Server, DHT11, dan Relay)

Hasil Pengujian & Serial Monitor

Pada saat program diunggah, NodeMCU terhubung ke hotspot Wi-Fi dan mendapatkan alamat IP.



(Screenshoot Tampilan Serial Monitor)

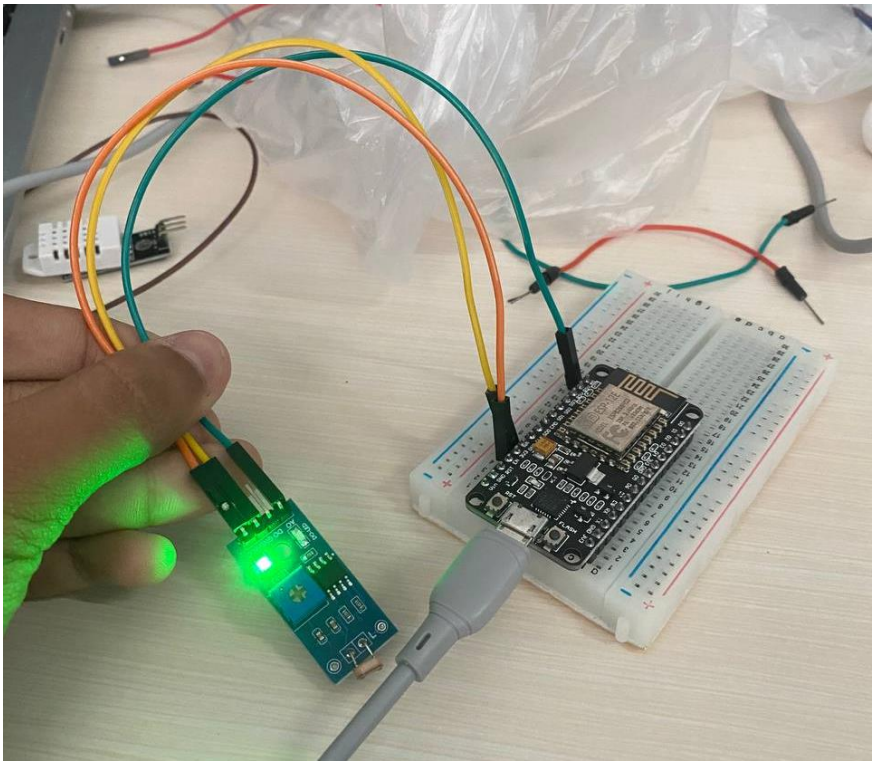
Analisis Program

- ESP8266WebServer server(80); mendengarkan lalu lintas pada *port* standar HTTP (80).
- Teknik *Raw String Literal* (R"rawliteral(...)rawliteral") digunakan untuk menyimpan antarmuka HTML secara efisien di memori flash (PROGMEM).
- Fungsi handleRoot() mengambil pembacaan suhu dari sensor DHT11, lalu mengganti placeholder %TEMPERATURE% secara dinamis dengan nilai suhu terkini menggunakan html.replace().
- Fungsi handleRelayOn() dan handleRelayOff() menerima navigasi URL /relay/on dan /relay/off, merubah *state* digital pin D6, lalu melakukan *redirect* (HTTP 303) kembali ke halaman utama /.

2.2 Praktikum 2: NodeMCU sebagai HTTP Client (Meminta Layanan)

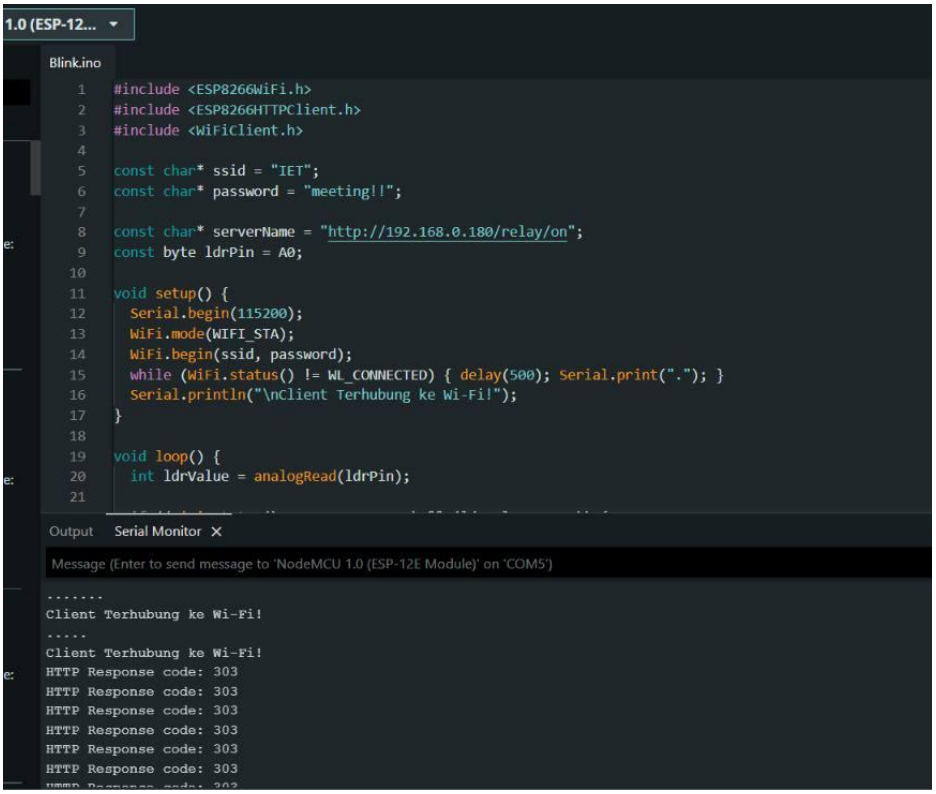
Rangkaian & Skematik

- **LDR Pin: Terhubung langsung ke Pin A0 NodeMCU, dengan kaki VCC ke 3V3 dan GND ke GND (menggunakan modul sensor LDR *built-in* / pembagi tegangan internal).**



(rangkaian NodeMCU Client dan sensor LDR)

Hasil Pengujian & Serial Monitor
Ketika LDR ditutup hingga kondisi gelap (ldrValue > 300), NodeMCU Client secara aktif membuat koneksi HTTP ke URL http://<IP_SERVER_PRAKTIKUM_1>/relay/on.



(Hasil Serial Monitor)

Analisis Program

- HTTPClient http; dan WiFiClient client; digunakan untuk menginisialisasi pustaka pembuat *request* HTTP.
- http.begin(client, serverName); mengarahkan permintaan ke alamat URL server target.
- Executing http.GET() mengirimkan pemicu pemicuan Relay jarak jauh tanpa interaksi langsung pengguna melalui antarmuka web.
- delay(10000); dipasang untuk mencegah kondisi *flooding/DDoS* request ke Server saat LDR berada dalam kondisi gelap terus-menerus.

2.3 JAWABAN LATIHAN

Latihan 1: Pemahaman Jaringan Mode Station (STA) vs Access Point (AP)

Soal: Mengapa kita menggunakan mode jaringan *Station (STA)* pada kedua praktikum di atas, dan bukan mode *Access Point (AP)*?

Jawaban:

Mode **Station (STA)** digunakan agar kedua NodeMCU (Server dan Client) berada di dalam satu infrastruktur jaringan Wi-Fi lokal (*subnet*) yang sama. Dengan terhubung ke sebuah *Router/Hotspot* sentral:

1. Kedua NodeMCU serta perangkat pengguna (Laptop/HP) dapat saling berkomunikasi secara simultan.
2. Jika menggunakan mode **Access Point (AP)**, NodeMCU Server hanya memancarkan jaringan lokalnya sendiri. Hal ini menyulitkan NodeMCU Client jika harus terhubung ke server sekaligus mempertahankan koneksi ke jaringan lain, atau jika terdapat banyak node yang perlu saling berkomunikasi dalam satu arsitektur IoT yang terintegrasi.

Latihan 2: Modifikasi Antarmuka & Telemetri (Kelembapan & Toggle Button)

Soal: Tuliskan potongan baris kode HTML dan fungsi handleRoot() yang telah dimodifikasi!

Jawaban:

1. Potongan Kode HTML

```
<div class="sensor-box">
  <p>Suhu Saat Ini: <strong>%TEMPERATURE%</strong> Celcius</p>
  <p>Kelembapan: <strong>%HUMIDITY%</strong> %%</p>
</div>
<h2>Kendali Relay</h2>
%RELAY_BUTTON%
```

2. Potongan Fungsi handleRoot():

```
void handleRoot() {
  String html = index_html;
  float t = dht.readTemperature();
  float h = dht.readHumidity();

  // Replace Temperature
  if (isnan(t)) {
    html.replace("%TEMPERATURE%", "--");
  } else {
    html.replace("%TEMPERATURE%", String(t));
  }

  // Replace Humidity
  if (isnan(h)) {
    html.replace("%HUMIDITY%", "--");
  } else {
    html.replace("%HUMIDITY%", String(h));
  }
```

```
// Toggle Button Dynamic Generation
int relayState = digitalRead(relayPin);
String buttonHtml = "";
if (relayState == LOW) { // Asumsi Relay Active HIGH atau kondisi OFF
  buttonHtml = "<a href=\"/relay/on\"><button class=\"btn-on\">NYALAKAN (ON)</butt
} else {
  buttonHtml = "<a href=\"/relay/off\"><button class=\"btn-off\">MATIKAN (OFF)</bu
}
html.replace("%RELAY_BUTTON%", buttonHtml);

server.send(200, "text/html", html);
}
```

Latihan 3: Analisis Kode Client Saat Server Offline

Soal: Analisislah apa yang akan dicetak oleh *Serial Monitor* Anda jika ternyata NodeMCU *Server* mendadak kehilangan daya (*offline*)?

Jawaban:
Jika NodeMCU Server mendadak *offline* atau kehilangan daya:

1. Fungsi http.GET() pada NodeMCU Client tidak akan mendapatkan balasan/respon HTTP dari server.
2. Variabel httpResponseCode akan bernilai **negatif** (misalnya -1 yang menandakan HTTPC_ERROR_CONNECTION_REFUSED atau HTTPC_ERROR_CONNECTION_FAILED).
3. Pada *Serial Monitor Client* akan tercetak:

HTTP Response code: -1 (atau kode eror internal HTTP Client library lainnya).

Latihan 4: Pengembangan Algoritma Klien Pendeteksi Suhu

Soal: Modifikasi *conditional block* if di Praktikum 2 sehingga *Client* mengirim permintaan (GET Request) **HANYA JIKA** sensor DHT membaca suhu melebihi 35°C!

Jawaban:

```
#include <DHT.h>
const byte dhtPin = 2; // Pin D4
DHT dht(dhtPin, DHT11);

// Di dalam fungsi loop():
void loop() {
  float temp = dht.readTemperature();

  if ((WiFi.status() == WL_CONNECTED) && (!isnan(temp)) && (temp > 35.0)) {
    WiFiClient client;
    HTTPClient http;

    http.begin(client, serverName);
    int httpResponseCode = http.GET();

    Serial.print("Suhu Tinggi Terdeteksi: ");
    Serial.print(temp);
    Serial.print(" C. HTTP Response code: ");
    Serial.println(httpResponseCode);

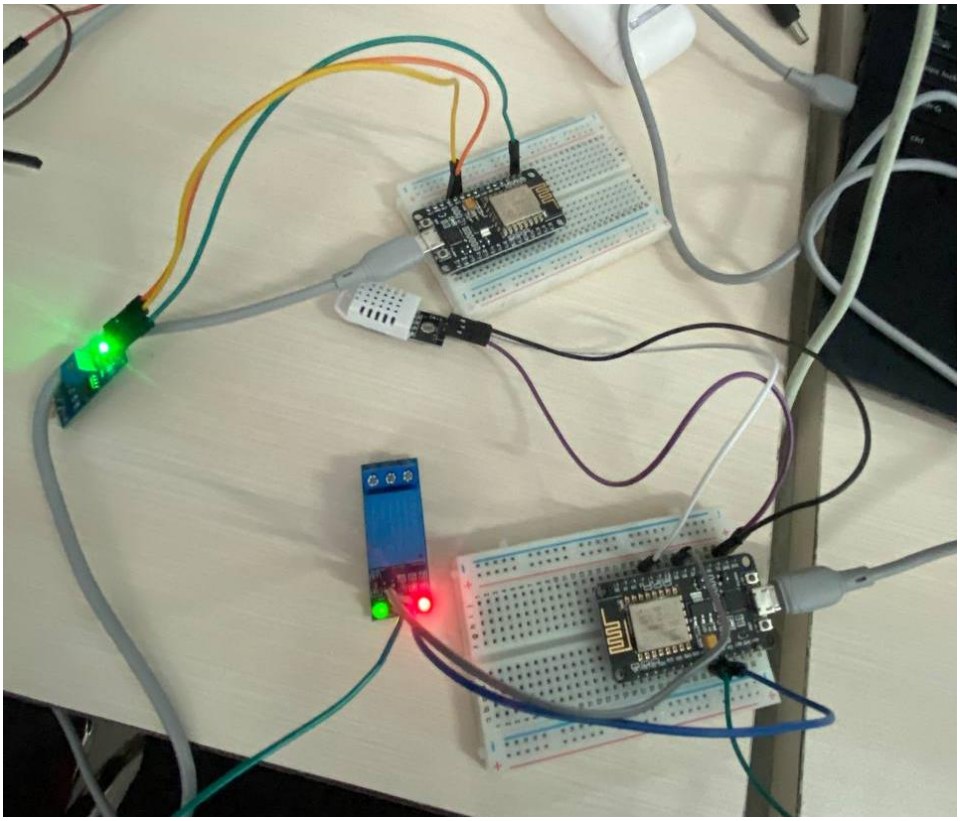
    http.end();
    delay(10000); // Cooldown agar tidak memborbardir server
  }
```

2.4 TUGAS: PENGEMBANGAN WEB SERVER CERDAS & ANALISIS ARSITEKTUR

2.4.1 Pelaksanaan Modifikasi (Auto-Refresh Meta Tag)

Penambahan sintaks <meta http-equiv="refresh" content="5"> disisipkan di dalam tag <head> pada variabel index_html:

```
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta http-equiv="refresh" content="5">
  <title>IoT Dashboard</title>
  ...
```



(Foto Rangkaian Tugas)

Link GDrive Video Tugas: https://drive.google.com/drive/folders/1WtR8nGxvkDiaizxg7n-PgOCGQFkf6cY1

2.4.2 Evaluasi & Analisis Arsitektur HTTP Konvensional

Hasil Observasi & Evaluasi Logis:

1. **Masalah *Flickering* (Layar Berkedip):** Saat interval 5 detik tercapai, *browser* melakukan *reload* total terhadap seluruh elemen DOM HTML. Hal ini menyebabkan layar berkedip putih (*flicker*) yang menggangu kenyamanan antarmuka pengguna.
2. **Inefisiensi *Bandwidth* & *Overhead* Jaringan:** Setiap 5 detik, *browser* meminta ulang seluruh dokumen HTML, *CSS style*, dan struktur teks lengkap dari NodeMCU Server, meskipun data yang berubah hanyalah angka variabel suhu/kelembapan. Hal ini membuang resource memori dan pemrosesan pada mikrokontroler.
3. **Potensi *Race Condition* / Bentrokan Interaksi:** Jika pengguna menekan tombol kendali Relay persis di detik ke-5 (saat proses *auto-refresh* terjadi), *request* tombol dapat dibatalkan atau terganggu oleh proses muat ulang halaman.
4. **Kesimpulan Arsitektur:** Metode *Meta Refresh* HTTP konvensional kurang ideal untuk *Real-Time Dashboard* IoT modern. Untuk mengatasi keterbatasan ini, arsitektur berbasis **AJAX / WebSockets / MQTT** lebih direkomendasikan karena dapat memperbarui data sensor secara asinkron tanpa memuat ulang seluruh halaman web.

BAB III
Kesimpulan

1. NodeMCU ESP8266 berhasil dikonfigurasi dalam mode **Station (STA)** untuk terhubung ke jaringan Wi-Fi lokal.
2. Implementasi **Web Server** pada NodeMCU memungkinkan penyajian halaman web dinamis berbasis HTML untuk pembacaan sensor dan kontrol actuator (*relay*).
3. Implementasi **HTTP Client** terbukti mampu memicu aksi pada perangkat lain secara jarak jauh menggunakan protokol HTTP GET.